Multivariate Normal Distributions

Covariance Geometry and Eigenfaces

CSE 534

Goal: Model several related continuous values at once

From one variable to many

Lesson 6: one continuous value at a time — $\mathcal{N}(\mu, \sigma^2)$.

Today: several related values together, stored as a *vector*.

- A biscuit tin: height X_1 and diameter X_2
- An image: one pixel intensity per position
- A latent code: many coordinates describing one generated sample

New tool: the *covariance matrix* $\boldsymbol{\Sigma}$ — records the spread of each measurement and how pairs of measurements move together.

Two independent measurements

Suppose height X_1 and diameter X_2 are unrelated — knowing one tells you nothing about the other.

Each is its own 1D normal, exactly as in Lesson 6:

$$X_1 \sim \mathcal{N}(\mu_1, \sigma_1^2), \qquad X_2 \sim \mathcal{N}(\mu_2, \sigma_2^2)$$

Joint probability: multiply the two densities — the ordinary product rule for independent events.

$$p(x_1, x_2) = \frac{1}{\sqrt{2\pi\sigma_1^2}} \exp\left(-\frac{(x_1-\mu_1)^2}{2\sigma_1^2}\right) \cdot \frac{1}{\sqrt{2\pi\sigma_2^2}} \exp\left(-\frac{(x_2-\mu_2)^2}{2\sigma_2^2}\right)$$

Combining into one exponential

Since $\exp(a)\exp(b) = \exp(a+b)$, the product of the two densities collapses into a single exponential:

$$p(x_1, x_2) = \frac{1}{2\pi\sigma_1\sigma_2} \exp\left(-\frac{1}{2}\left[\frac{(x_1-\mu_1)^2}{\sigma_1^2} + \frac{(x_2-\mu_2)^2}{\sigma_2^2}\right]\right)$$

Combining into one exponential

Since $\exp(a)\exp(b) = \exp(a+b)$, the product of the two densities collapses into a single exponential:

$$p(x_1, x_2) = \frac{1}{2\pi\sigma_1\sigma_2} \exp\left(-\frac{1}{2}\left[\frac{(x_1-\mu_1)^2}{\sigma_1^2} + \frac{(x_2-\mu_2)^2}{\sigma_2^2}\right]\right)$$

Let $v_1 = x_1 - \mu_1$ and $v_2 = x_2 - \mu_2$ — how far each coordinate sits from its mean. Substituting:

$$p(x_1, x_2) = \frac{1}{2\pi\sigma_1\sigma_2} \exp\left(-\frac{1}{2}\left[\frac{v_1^2}{\sigma_1^2} + \frac{v_2^2}{\sigma_2^2}\right]\right)$$

The exponent as a matrix product

The bracketed sum is the only part of the density that depends on $\mathbf{x}$:

$$\frac{v_1^2}{\sigma_1^2} + \frac{v_2^2}{\sigma_2^2}$$

This equals the quadratic form $\mathbf{v}^T\boldsymbol{\Sigma}^{-1}\mathbf{v}$, where $\boldsymbol{\Sigma}^{-1}$ is diagonal with $1/\sigma_1^2$ and $1/\sigma_2^2$ on it:

The exponent as a matrix product

The bracketed sum is the only part of the density that depends on $\mathbf{x}$:

$$\frac{v_1^2}{\sigma_1^2} + \frac{v_2^2}{\sigma_2^2}$$

This equals the quadratic form $\mathbf{v}^T\boldsymbol{\Sigma}^{-1}\mathbf{v}$, where $\boldsymbol{\Sigma}^{-1}$ is diagonal with $1/\sigma_1^2$ and $1/\sigma_2^2$ on it:

$$\mathbf{v}^T\boldsymbol{\Sigma}^{-1}\mathbf{v} = \begin{pmatrix} v_1 & v_2 \end{pmatrix} \begin{pmatrix} \sigma_1^{-2} & 0 \\ 0 & \sigma_2^{-2} \end{pmatrix} \begin{pmatrix} v_1 \\ v_2 \end{pmatrix}$$

The exponent as a matrix product

The bracketed sum is the only part of the density that depends on $\mathbf{x}$:

$$\frac{v_1^2}{\sigma_1^2} + \frac{v_2^2}{\sigma_2^2}$$

This equals the quadratic form $\mathbf{v}^T\boldsymbol{\Sigma}^{-1}\mathbf{v}$, where $\boldsymbol{\Sigma}^{-1}$ is diagonal with $1/\sigma_1^2$ and $1/\sigma_2^2$ on it:

$$\begin{aligned}\mathbf{v}^T\boldsymbol{\Sigma}^{-1}\mathbf{v} &= \begin{pmatrix} v_1 & v_2 \end{pmatrix} \begin{pmatrix} \sigma_1^{-2} & 0 \\ 0 & \sigma_2^{-2} \end{pmatrix} \begin{pmatrix} v_1 \\ v_2 \end{pmatrix} \\ &= \begin{pmatrix} v_1 & v_2 \end{pmatrix} \begin{pmatrix} \sigma_1^{-2} v_1 \\ \sigma_2^{-2} v_2 \end{pmatrix}\end{aligned}$$

The exponent as a matrix product

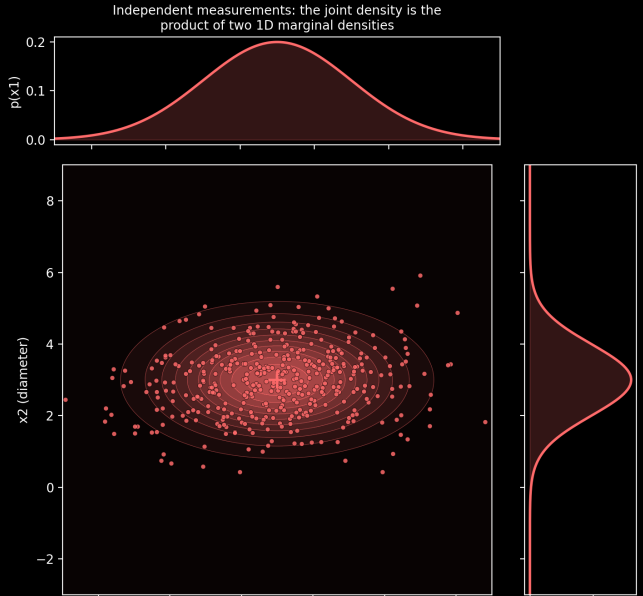
The bracketed sum is the only part of the density that depends on $\mathbf{x}$:

$$\frac{v_1^2}{\sigma_1^2} + \frac{v_2^2}{\sigma_2^2}$$

This equals the quadratic form $\mathbf{v}^T\mathbf{\Sigma}^{-1}\mathbf{v}$, where $\mathbf{\Sigma}^{-1}$ is diagonal with $1/\sigma_1^2$ and $1/\sigma_2^2$ on it:

$$\begin{aligned}
\mathbf{v}^T\mathbf{\Sigma}^{-1}\mathbf{v} &= \begin{pmatrix} v_1 & v_2 \end{pmatrix} \begin{pmatrix} \sigma_1^{-2} & 0 \\ 0 & \sigma_2^{-2} \end{pmatrix} \begin{pmatrix} v_1 \\ v_2 \end{pmatrix} \\
&= \begin{pmatrix} v_1 & v_2 \end{pmatrix} \begin{pmatrix} \sigma_1^{-2} v_1 \\ \sigma_2^{-2} v_2 \end{pmatrix} \\
&= \frac{v_1^2}{\sigma_1^2} + \frac{v_2^2}{\sigma_2^2}
\end{aligned}$$

Independent measurements: what it looks like



Correlated dimensions: entry by entry

Now suppose taller biscuit tins also tend to be wider — a single variance per dimension is no longer enough.
Let $v_i = X_i - \mu_i$ for each of d measurements. The covariance matrix $\mathbf{\Sigma}$ is a $d \times d$ matrix of expected products:

$$\mathbf{\Sigma} = \mathbb{E}\begin{pmatrix} v_1^2 & v_1 v_2 & \cdots & v_1 v_d \\ v_2 v_1 & v_2^2 & \cdots & v_2 v_d \\ \vdots & \vdots & \ddots & \vdots \\ v_d v_1 & v_d v_2 & \cdots & v_d^2 \end{pmatrix}$$

Diagonal: $\mathbb{E}[v_i^2] = \mathrm{Var}(X_i)$ — the variance of each measurement, alone.
Off-diagonal: $\mathbb{E}[v_i v_j]$ — positive means the two rise together, negative means one falls as the other rises, near zero means little relationship.

Reminder: outer products

An **outer product** multiplies a column vector by a row vector, producing a matrix — not a number.

For a d-dimensional column vector $\mathbf{v}$:

$$\mathbf{v}\mathbf{v}^T = \begin{pmatrix} v_1 \\ v_2 \\ \vdots \\ v_d \end{pmatrix} \begin{pmatrix} v_1 & v_2 & \cdots & v_d \end{pmatrix} = \begin{pmatrix} v_1^2 & v_1 v_2 & \cdots & v_1 v_d \\ v_2 v_1 & v_2^2 & \cdots & v_2 v_d \\ \vdots & \vdots & \ddots & \vdots \\ v_d v_1 & v_d v_2 & \cdots & v_d^2 \end{pmatrix}$$

Shape check: $(d \times 1)(1 \times d) = (d \times d)$ — the opposite of a dot product $\mathbf{v}^T\mathbf{v}$, which is $(1 \times d)(d \times 1) = (1 \times 1)$, a scalar.

Correlated dimensions: compact notation

The matrix from the previous slides has a compact form, valid for any number of dimensions:

$$\boldsymbol{\Sigma} = \mathbb{E}\left[(\mathbf{X} - \boldsymbol{\mu})(\mathbf{X} - \boldsymbol{\mu})^T\right]$$

Why this works: $(\mathbf{X} - \boldsymbol{\mu})$ is a d-dimensional column vector; multiplying it by its own transpose is exactly the outer product from the previous slide, with $\mathbf{v} = \mathbf{X} - \boldsymbol{\mu}$.

When every off-diagonal entry is zero, this reduces exactly to the independent case from earlier.

Estimating Σ from data: the matrix $\mathbf{V}$

Mean-center each sample: $\mathbf{v}^{(i)} = \mathbf{x}^{(i)} - \boldsymbol{\mu} = \left(v_1^{(i)}, v_2^{(i)}\right)$.

Stack the centered samples as *rows* of an $n \times 2$ matrix $\mathbf{V}$ — the same row-per-sample layout `numpy`, `pandas`, and `scikit-learn` use:

$$\mathbf{V} = \begin{pmatrix} v_1^{(1)} & v_2^{(1)} \\ v_1^{(2)} & v_2^{(2)} \\ \vdots & \vdots \\ v_1^{(n)} & v_2^{(n)} \end{pmatrix}$$

— one row per sample, one column per coordinate

Estimating Σ from data: the formula

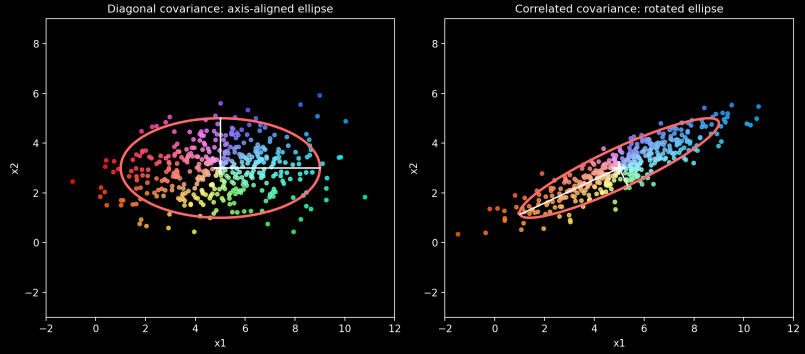
With $\mathbf{V}$ as on the previous slide, the covariance matrix is the averaged matrix product $\mathbf{V}^T\mathbf{V}$ — equivalently, the average of n outer products, one per sample:

$$\mathbf{\Sigma} = \frac{1}{n}\mathbf{V}^T\mathbf{V} = \frac{1}{n}\sum_{i=1}^{n}\mathbf{v}^{(i)}\left(\mathbf{v}^{(i)}\right)^T = \frac{1}{n}\sum_{i=1}^{n}\begin{pmatrix} v_1^2 & v_1 v_2 \\ v_1 v_2 & v_2^2 \end{pmatrix}$$

Diagonal entries: average squared deviation of each coordinate. Off-diagonal: average product of the two coordinates — the covariance.

The general multivariate density

$$p(\mathbf{x};\boldsymbol{\mu},\boldsymbol{\Sigma}) = \frac{1}{(2\pi)^{d/2}|\boldsymbol{\Sigma}|^{1/2}} \exp\left(-\frac{1}{2}(\mathbf{x}-\boldsymbol{\mu})^T\boldsymbol{\Sigma}^{-1}(\mathbf{x}-\boldsymbol{\mu})\right)$$



Left: diagonal covariance, axis-aligned (same as the independent case). Right: a positive off-diagonal covariance tilts the ellipse. Black lines mark the principal axes.

A recipe for sampling correlated Gaussians

Having seen how to estimate $\mathbf{\Sigma}$ from data, and using the ordinary sample mean for μ — here is a recipe for the opposite problem: generating correlated samples from any chosen μ and $\mathbf{\Sigma}$.

Start with standard normal noise:

$$\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$$

Transform: find a matrix $\mathbf{A}$ with $\mathbf{A}\mathbf{A}^T = \mathbf{\Sigma}$, then

$$\mathbf{x} = \mu + \mathbf{A}\mathbf{z}$$

Same reparameterization as $\sigma z + \mu$ from Lesson 6, with $\mathbf{A}$ replacing σ.

Building A: scale and rotate

Using two dimensions to visualize the idea — everything here extends directly to d dimensions.

Scale matrix S — desired standard deviations on the diagonal:

$$\mathbf{S} = \begin{pmatrix} \sigma_1 & 0 \\ 0 & \sigma_2 \end{pmatrix}$$

Rotation $\mathbf{R}(\theta)$ — turns the axes toward the directions you want the ellipse to stretch:

$$\mathbf{R}(\theta) = \begin{pmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{pmatrix}$$

Combine: $\mathbf{A} = \mathbf{RS}$

Why $\mathbf{R}^{-1} = \mathbf{R}^T$

Write $\mathbf{R} = \begin{pmatrix} \mathbf{r}_1 & \mathbf{r}_2 \end{pmatrix}$ — its columns are the new axis directions.

Each column has length 1: $\mathbf{r}_i^T \mathbf{r}_i = \cos^2\theta + \sin^2\theta = 1$

Different columns are perpendicular: $\mathbf{r}_1^T \mathbf{r}_2 = -\cos\theta\sin\theta + \sin\theta\cos\theta = 0$

So:

$$\mathbf{R}^T\mathbf{R} = \begin{pmatrix} \mathbf{r}_1^T\mathbf{r}_1 & \mathbf{r}_1^T\mathbf{r}_2 \\ \mathbf{r}_2^T\mathbf{r}_1 & \mathbf{r}_2^T\mathbf{r}_2 \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} = \mathbf{I}$$

$\mathbf{R}^T\mathbf{R} = \mathbf{I}$ is exactly the definition of $\mathbf{R}^{-1} = \mathbf{R}^T$ — the next slide confirms this holds the other way too.

Why $\mathbf{R}^{-1} = \mathbf{R}^T$, the other way

The same argument works on *rows* instead of columns: write $\mathbf{R} = \begin{pmatrix} \mathbf{r}_1^T \\ \mathbf{r}_2^T \end{pmatrix}$ using its rows this time.

Rows are also unit length and perpendicular to each other, so:

$$\mathbf{R}\mathbf{R}^T = \begin{pmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{pmatrix} \begin{pmatrix} \cos\theta & \sin\theta \\ -\sin\theta & \cos\theta \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} = \mathbf{I}$$

$\mathbf{R}^T\mathbf{R} = \mathbf{R}\mathbf{R}^T = \mathbf{I}$: since $\mathbf{R}$ is square, a left inverse is automatically a two-sided inverse.

Applying A: scale first, then rotate

Reading $\mathbf{Az} = \mathbf{R}(\mathbf{Sz})$ right to left shows the actual order of operations:

$$\mathbf{Az} = \mathbf{RSz} = \mathbf{R}\underbrace{(\mathbf{Sz})}_{\text{scale first}}$$

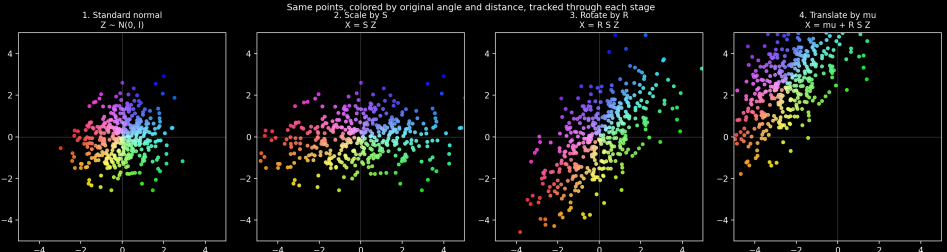
Scale first, in the axis-aligned frame where scaling means something concrete — stretching along a known direction — and only then rotate into the orientation you actually want.

Analogy: if getting from room A to room C means walking through room B first, retracing your steps means leaving room B, then room A — not the other way around.

Confirming the covariance: transposing a product reverses the order of its factors:

$$\mathbf{AA}^T = (\mathbf{RS})(\mathbf{RS})^T = \mathbf{RSS}^T\mathbf{R}^T = \mathbf{RS}^2\mathbf{R}^T$$

The recipe, visualized

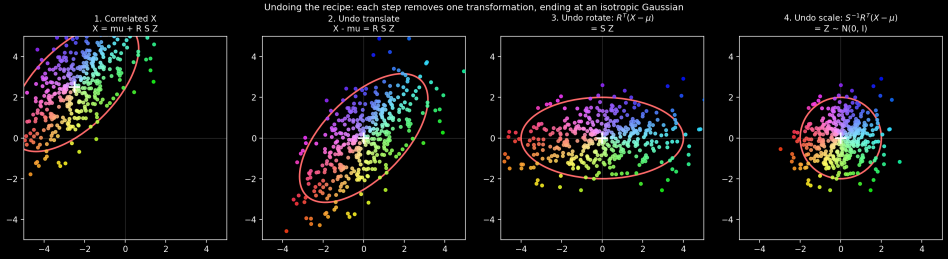


Standard-normal samples (1) are stretched by **S** (2), rotated by **R** (3), and shifted by $\boldsymbol{\mu}$ (4) — producing a correlated Gaussian with a chosen center, spread, and orientation.

Undoing the recipe: whitening

The recipe runs in reverse too: given correlated $\mathbf{x}$, undo each step to recover standard-normal $\mathbf{z}$.

$$\mathbf{z} = \mathbf{S}^{-1}\mathbf{R}^T(\mathbf{x} - \mu)$$



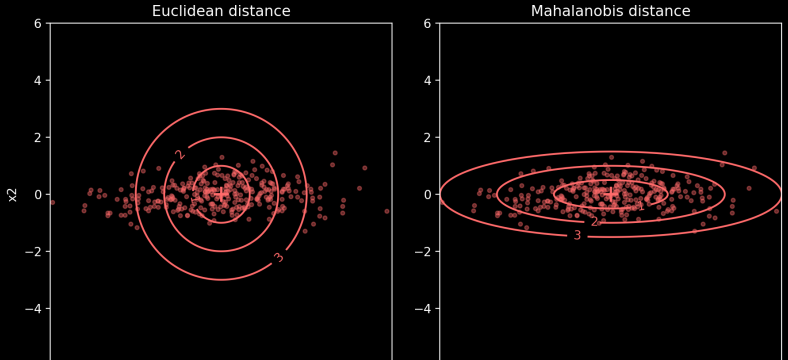
Undo the translation, then the rotation ($\mathbf{R}^T$), then the scale ($\mathbf{S}^{-1}$) — the ellipse un-tilts and un-stretches at each step, ending at an isotropic circle.

Mahalanobis distance

Ordinary (Euclidean) distance from $\boldsymbol{\mu}$ treats every direction equally — misleading once the ellipse is tilted or stretched.

Mahalanobis distance is exactly the Euclidean distance of the whitened point $\mathbf{z} = \mathbf{S}^{-1}\mathbf{R}^T(\mathbf{x} - \boldsymbol{\mu})$ from the origin:

$$D_M(\mathbf{x}) = \|\mathbf{z}\| = \sqrt{(\mathbf{x} - \boldsymbol{\mu})^T \boldsymbol{\Sigma}^{-1} (\mathbf{x} - \boldsymbol{\mu})}$$



Principal component axes

The directions $\mathbf{R}$ points toward are the **principal component axes** of $\mathbf{\Sigma}$ — the directions of greatest and least spread, always perpendicular.

Formally, these are the **eigenvectors** of $\mathbf{\Sigma}$; the **eigenvalues** give the variance along each direction (σ_1^2, σ_2^2).

$$\mathbf{S}\mathbf{S}^T = \begin{pmatrix} \sigma_1^2 & 0 \\ 0 & \sigma_2^2 \end{pmatrix}$$

Key idea: every covariance matrix is really a diagonal one (independent measurements), viewed from a rotated angle.

Fitting a Gaussian is PCA

Fitting a multivariate Gaussian means estimating $\boldsymbol{\mu}$ and $\boldsymbol{\Sigma}$.

The mean is just the sample mean, exactly as in the 1D case — only now each $\mathbf{x}_i$ is a vector:

$$\hat{\boldsymbol{\mu}} = \frac{1}{n}\sum_{i=1}^{n}\mathbf{x}_i$$

The covariance is $\hat{\boldsymbol{\Sigma}} = \frac{1}{n}\mathbf{V}^T\mathbf{V}$, shown a few slides back.

What you usually want: the principal axes and variances along them — the eigenvectors and eigenvalues of $\boldsymbol{\Sigma}$.

```
eigenvalues, eigenvectors = numpy.linalg.eigh(Sigma)
std_devs = numpy.sqrt(eigenvalues)  # diagonal of S
R = eigenvectors  # columns = axes
```

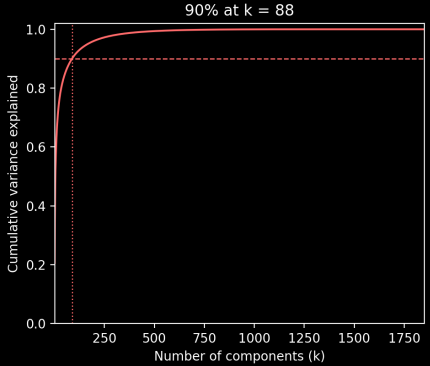
Eigenfaces: a generative model for images

Everything above still works when each "point" is an entire image.

- Flatten a grayscale face photo into one long vector $\mathbf{x}$ — one entry per pixel
- A collection of faces becomes a cloud of points in a very high-dimensional space
- Fit a multivariate Gaussian using nothing more than the mean and covariance formulas above

This is the idea behind **eigenfaces** (Turk and Pentland, 1991).

Mean face, eigenfaces, and variance explained



Left: the mean face $\hat{\mu}$. Middle: the top 16 eigenfaces — the principal axes of $\mathbf{\Sigma}$, reshaped back into images. Right: cumulative variance explained.

Sampling new faces

Same sampling recipe from equation 7.9, applied to images:

$$\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}), \qquad \mathbf{x} = \hat{\mu} + \mathbf{A}\mathbf{z}$$

Each entry of $\mathbf{z}$ is a random weight used to create a mixture of the eigenfaces. Adding the weighted eigenfaces to the mean face produces a new synthetic face.

In practice: keep only the first few dozen or hundred eigenfaces (largest variance) — the rest mostly capture noise.

This is a genuine **generative model**: sampling produces new examples, not just describing existing ones.

Sampled faces vs. real faces

Sampled faces (top 150 eigenfaces)

Real faces

Left: nine faces sampled using only the top 150 eigenfaces ($\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$, $\mathbf{x} = \hat{\mu} + \mathbf{R}(\mathbf{S}\mathbf{z})$), with $\mathbf{z}$ truncated to ± 0.8 standard deviations so every sample stays in the dense, realistic-looking part of the distribution instead of the noisier tails. Right: nine real faces from the dataset, for comparison.

PCA for dimension reduction

Each face is a point in a 1850-pixel space — far too many dimensions to plot or reason about directly.

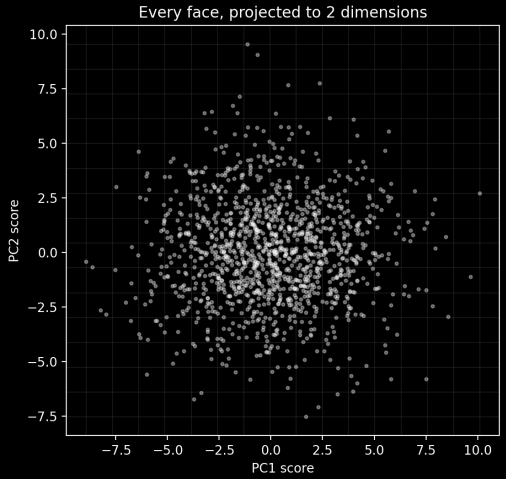
Dimension reduction: keep only the coordinates along the top few principal axes, and drop the rest.

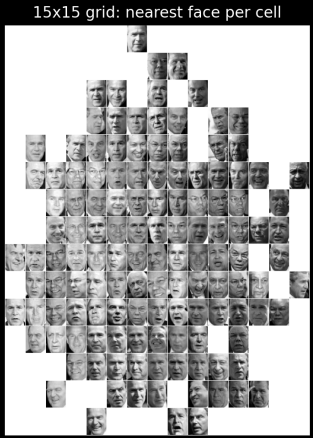
Projecting onto just the **top 2** eigenfaces gives every face a $(\text{PC1}, \text{PC2})$ score — two numbers standing in for 1850 pixel values:

$$\text{score} = \mathbf{v}^T \begin{pmatrix} \mathbf{r}_1 & \mathbf{r}_2 \end{pmatrix}$$

Nearby scores should belong to similar-looking faces — the next slide checks whether that is actually true.

Visualizing the top 2 dimensions





Left: every face's (PC1, PC2) score, with a 15×15 grid overlaid. Right: the face closest to each grid cell's center — blank where no face landed in that cell.

Beyond PCA: other ways to see your data

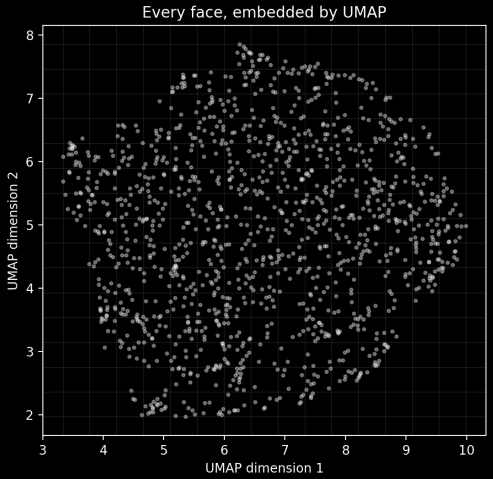
PCA is not the only way to compress data down to 2 dimensions for visualization — and it has a real limitation: it only looks for **straight-line** directions of spread.

Two popular alternatives instead try to preserve which points are **near each other**, even if that means bending the data into a different shape:

- **UMAP** — Uniform Manifold Approximation and Projection
- **t-SNE** — t-distributed Stochastic Neighbor Embedding

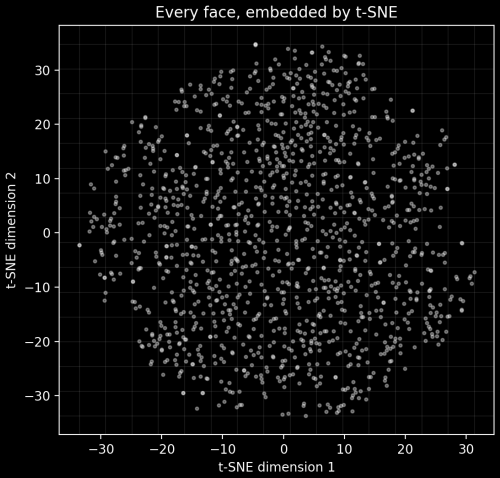
Both are useful for exploring whether your data has clusters or structure that a straight-line method like PCA might miss.

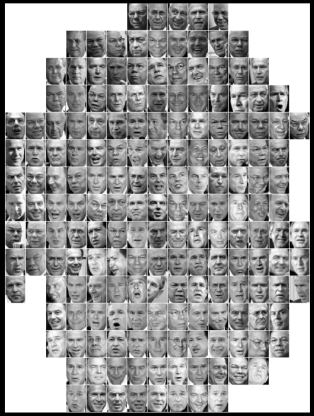
UMAP: same faces, a different embedding



The same 15×15 nearest-face-per-cell grid, but the 2D coordinates come from UMAP instead of the top 2 principal components.

t-SNE: same faces, another embedding





The same 15×15 nearest-face-per-cell grid, using t-SNE's 2D coordinates instead.

Coming up next

Next: Tool Use, Retrieval, and Agentic Loops.

Further reading:

- PRML (Bishop), Section 2.3
- 3Blue1Brown: Linear transformations and matrices
- Gilbert Strang, MIT 18.06 — Lectures 1–3 and 21 (eigenvalues and eigenvectors)
- Turk and Pentland, "Eigenfaces for Recognition," 1991
- McInnes, Healy, and Melville, "UMAP: Uniform Manifold Approximation and Projection," 2018
- van der Maaten and Hinton, "Visualizing Data using t-SNE," 2008